FUNCTIONAL HORROR PROPS

Interactivity & Integration Guide

300 animated, interactive horror props — one drop-in component, no Animator, no setup. Works in Built-in, URP and HDRP.

Contents

1. The one component you need **3**

1.1 Public API 3

1.2 What “active” does, per motion type 3

2. Wiring it to gameplay **3**

2.1 Player “press E to interact” (raycast) 3

2.2 Trigger volume (open as the player approaches) 4

2.3 Timeline / cutscene / scripted scare 4

2.4 Editor preview without play mode 4

3. Vehicles: FunctionalHorrorVehicle **4**

4. Cross-pipeline: Built-in / URP / HDRP **4**

5. Demo scenes **5**

6. Quick reference **5**

1. The one component you need

Every animated prop in this pack is driven by one small runtime component, **FunctionalHorrorProp** (namespace `FunctionalHorrorProps`). There is no Animator, no animation clip, no input package and no setup required — drop a prefab in your scene and call one method. Static props (walls, floors, a plain stool) do not carry it.

1.1 Public API

```
using FunctionalHorrorProps;

FunctionalHorrorProp prop = go.GetComponent<FunctionalHorrorProp>();

prop.Activate();      // open the door / start the motor / begin flicker
prop.Deactivate();    // close / coast to a stop / stop flicker
prop.Toggle();        // flip between the two
bool open   = prop.IsActive;         // current state
bool canMove= prop.HasInteraction;   // false for static props
string name = prop.DisplayName;      // e.g. "Medicine cabinet"
```

That is the whole interface. Doors and machinery start **inactive**, so a scene stays still and tense until you cue it.

1.2 What “active” does, per motion type

A prop can carry one or more motions. The mode decides what `Activate()` means:

Mode	Active behaviour	Example props
Rotate	Swings open to a set angle and stays open; Deactivate() closes it	doors, gates, cabinet lids
Slide	Slides open along an axis	drawers, mortuary trays, sliding gates
Spin	Spins continuously while active; coasts to a stop	fans, wheels, drums, turntables
Rock / Oscillate	Sways back and forth while active	hanging censer, pendulums, signals
Flicker	Pulses the prop's emission + light irregularly	lamps, candles, sconces, tesla coil

Flicker props carry their own light (a child “Controlled light”) and an emissive material, so turning them on lights the room.

2. Wiring it to gameplay

2.1 Player “press E to interact” (raycast)

```
void Interact(Camera cam) {
  if (Physics.Raycast(cam.transform.position, cam.transform.forward, out var hit, 3f)) {
```

```
    var prop = hit.collider.GetComponentInParent<FunctionalHorrorProp>();
    if (prop != null && prop.HasInteraction) prop.Toggle();
  }
}
```

Every prop ships with mesh colliders, so raycasts hit them out of the box.

2.2 Trigger volume (open as the player approaches)

```
public class OpenOnEnter : MonoBehaviour {
  [SerializeField] FunctionalHorrorProp target;
  void OnTriggerEnter(Collider o){ if (o.CompareTag("Player")) target.Activate(); }
  void OnTriggerExit (Collider o){ if (o.CompareTag("Player")) target.Deactivate(); }
}
```

2.3 Timeline / cutscene / scripted scare

Call `Activate()` / `Deactivate()` from a Signal receiver, a coroutine or an event — e.g. flip a whole ward of lamps to flicker on a cue:

```
foreach (var lamp in FindObjectsByType<FunctionalHorrorProp>(FindObjectsSortMode.None))
  if (lamp.DisplayName.Contains("lamp")) lamp.Activate();
```

2.4 Editor preview without play mode

`prop.ApplyPose(amount, seconds)` sets a deterministic pose (0 = closed, 1 = open; `seconds` drives spin / flicker / sway). Useful for thumbnails, tests, or posing a door half-open in a screenshot.

3. Vehicles: FunctionalHorrorVehicle

The derelict vehicles use a second drop-in component. One dial drives everything.

```
using FunctionalHorrorProps;

var car = vehicleRoot.GetComponent<FunctionalHorrorVehicle>();
car.speedKmh = 30f;        // wheels spin at the matching rate; tailpipe smokes harder
car.speedKmh = 0f;         // parked: wheels still, engine idles a thin wisp
car.engineRunning = false; // dead engine: no smoke
car.AutoWire();            // (re)discover wheels + tailpipe from the model
```

`speedKmh` (0–140) spins every wheel at the correct angular velocity for its radius and feeds a procedurally-built exhaust smoke particle system (no texture asset needed). It finds the wheels and tailpipe from the model's part names, so it works on every motorised vehicle in the pack; trailers and the bicycle spin their wheels only.

Menu shortcut: **Tools ■ Functional Horror Props ■ Rig Selected Vehicles (wheels + exhaust)** rigs a selection in one click; **Un-rig Selected Vehicles** reverses it. Tick **Run In Edit Mode** on the component to see it spin without entering play.

4. Cross-pipeline: Built-in / URP / HDRP

The pack is built with Built-in materials plus an auto-adapter:

- On import into a URP or HDRP project, materials regenerate for that pipeline automatically.
- If anything looks pink, run **Tools ■ Functional Horror Props ■ Fix Materials For This Pipeline** once.
- Lighting note: the demo scenes are lit for the Built-in pipeline (distance fog + standard lights). They open and render in URP and HDRP too; for the identical haze add that pipeline's fog (URP: a Volume with Fog; HDRP: a Volume with Fog + exposure). HDRP uses physical light units, so scale the demo lights up if you reuse them there.

5. Demo scenes

Under `Assets/FunctionalHorrorProps/Scenes/`:

- **Demo_NightShiftWard, Demo_OperatingRoom, Demo_Morgue, Demo_Chapel, Demo_Corridor, Demo_ExteriorNight** — lit horror vignettes you can open and walk through.
- **Demo_ShowcaseMachinery, Demo_ShowcaseDoors** — tight halls that line up the moving machinery and the doors/gates for a close look.

Open one, enter play mode, and call `Toggle()` on any prop (or use the included `HorrorPropsDemo` gallery browser) to see doors swing, drawers slide, lamps flicker and machinery turn.

6. Quick reference

```
prop.Activate();  prop.Deactivate();  prop.Toggle();    // FunctionalHorrorProp
prop.IsActive;    prop.HasInteraction;                  // state
prop.ApplyPose(amount, seconds);                        // deterministic editor pose
vehicle.speedKmh = 25f;  vehicle.engineRunning = true;  // FunctionalHorrorVehicle
```

Functional Horror Props — abandoned-hospital interactive prop pack.